

1. Logika cyfrowa

| Układ              | Formuła / sygnatura                                                 | Opis                                                                                                           |
|--------------------|---------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Sumatory           |                                                                     |                                                                                                                |
| Półsumator         | $s = x \oplus y$<br>$c = x \& y$                                    | Dodaje dwa bity. <b>s</b> to suma, <b>c</b> to bit przeniesienia (carry).                                      |
| Pełny sumator (FA) | $s = x \oplus y \oplus ci$<br>$co = x \& y \vee ci \& (x \oplus y)$ | Jak półsumator, ale przyjmuje też przeniesienie wejściowe <b>ci</b> .                                          |
| Sumator n-bitowy   | $n \times FA$                                                       | Kaskada (ripple-carry): wyjście $C_i$ trafia na wejście kolejnego FA. Ostatnie <b>co</b> to Carry Out całości. |

|                     |                         |                                                                                                                     |
|---------------------|-------------------------|---------------------------------------------------------------------------------------------------------------------|
| Układy kombinacyjne |                         |                                                                                                                     |
| Dekoder             | $n \rightarrow 2^n$     | Wejście koduje liczbę $k$ . Na wyjściu zapalony jest <b>tylko</b> bit nr $k$ .                                      |
| Multiplekser        | $2^n + n \rightarrow 1$ | $2^n$ bitów danych + $n$ bitów sterujących $S$ . Na wyjście przechodzi $S$ -ty bit danych.                          |
| ALU                 | $A, B, f \rightarrow C$ | Dekoder na bitach $f$ wybiera operację (np. $A + B$ , $A B$ , $A \& B$ ); multipleksowanie wyników bramkami AND/OR. |

|                             |                                  |                                                                                                                                                    |
|-----------------------------|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Układy sekwencyjne (pamięć) |                                  |                                                                                                                                                    |
| Przerzutnik S-R             | <b>S</b> - set, <b>R</b> - reset | Przechowuje 1 bit ( $Q$ ). <b>S=1</b> ustawia $Q = 1$ , <b>R=1</b> zeruje, <b>S=R=0</b> trzyma stan. Zbudowany z dwóch sprzężonych <b>NOR</b> -ów. |
| Sterowany poziomem          | aktywny gdy <b>CLK</b> = 1       | Wejścia <b>S</b> / <b>R</b> bramkowane AND-em z zegarem. Stan zmienia się tylko przy <b>CLK=1</b> .                                                |
| Sterowany zboczem           | zbocze 1 $\rightarrow$ 0         | Dwa przerzutniki poziomowe w kaskadzie (master-slave), drugi z zanegowanym zegarem. Stan przepisuje się w <b>momencie</b> zmiany zegara.           |

2. Bity, bajty i typy danych

| Typ    | char | short | int | long | float | double | void* |
|--------|------|-------|-----|------|-------|--------|-------|
| x86-64 | 1    | 2     | 4   | 8    | 4     | 8      | 8     |

3. Kodowanie liczb i konwersja

$$B2U(X) = \sum_{i=0}^{w-1} x_i 2^i \quad B2T(X) = -x_{w-1} 2^{w-1} + \sum_{i=0}^{w-2} x_i 2^i$$
$$T2U(x) = x < 0 ? x + 2^w : x \quad U2T(u) = u > TMax ? u - 2^w : u$$

**Skróty:** **B** = bity, **U** = unsigned, **T** = ze znakiem (kod U2). Stąd **B2U** = bity  $\rightarrow$  unsigned, podobnie **U2T**, **UMax**, **TMax**, **UAdd**, **TAdd**.

| Stała | Bity   | Wartość                          |
|-------|--------|----------------------------------|
| UMax  | 11...1 | $2^w - 1$                        |
| TMax  | 01...1 | $2^{w-1} - 1$ ( <b>INT_MAX</b> ) |
| TMin  | 10...0 | $-2^{w-1}$ ( <b>INT_MIN</b> )    |
| -1    | 11...1 | te same bity co UMax             |

**Promocja w C:** **unsigned** i **int** w jednym wyrażeniu/porównaniu ( $< > ==$ )  $\rightarrow$  **int** promowany do **unsigned** (bity bez zmian,  $-1 \rightarrow$  UMax).

4. Rozszerzanie, obcinanie i przesunięcia

|                                |                                               |
|--------------------------------|-----------------------------------------------|
| Rozszerz. 0 ( <b>zext</b> )    | <b>unsigned</b> : dopisuje 0 z góry           |
| Rozszerz. znak ( <b>sext</b> ) | <b>signed</b> : powiela bit znaku (MSB)       |
| $x \ll k$                      | $x \cdot 2^k$ (sgn./uns.)                     |
| $u \gg k$                      | $\lfloor u/2^k \rfloor$ — logiczne (zera)     |
| $x \gg k$                      | arytmetyczne (kopia MSB), zaokr. do $-\infty$ |
| $x/2^k$ do 0                   | $(x + (1 \ll k) - 1) \gg k$                   |
| Strength red.: $x * 24$        | $(x \ll 5) - (x \ll 3)$ .                     |

5. Operacje, overflow i endianness

|                                |                                                           |
|--------------------------------|-----------------------------------------------------------|
| UAdd                           | $(u + v) \bmod 2^w$ ; carry ignorowane                    |
| TAdd                           | bitowo = UAdd; różne znaki nie przepełniają               |
| Nadmiar (+)                    | $u, v > 0$ , a wynik $< 0$ (wynik $-2^w$ )                |
| Nadmiar (-)                    | $u, v < 0$ , a wynik $\geq 0$ (wynik $+2^w$ )             |
| Negacja U2                     | $\sim x = \sim x + 1$ ; $\sim TMin = TMin$ , $\sim 0 = 0$ |
| Wykr. nadmiaru ( $s = x + y$ ) | $((s \wedge x) \& (s \wedge y)) \gg (w - 1)$              |
| Maska / abs                    | $m = x \gg 31$ ; $abs = (x \wedge m) - m$                 |

**Pamięć:** adres wskazuje najmłodszy bajt słowa. Napis = **char[]** ASCII + **0x00** (niezależny od endianness).

| Schemat       | Najniższy adres | 0x0123    |
|---------------|-----------------|-----------|
| Big Endian    | MSB             | [01][123] |
| Little Endian | LSB             | [23][01]  |

## 6. Zmiennoprzecinkowe (IEEE 754)

$V = (-1)^s \times M \times 2^E$       Bias =  $2^{k-1} - 1$

| Format          | bity | s | exp (k)        | frac (n) |
|-----------------|------|---|----------------|----------|
| Single (float)  | 32   | 1 | 8 (Bias=127)   | 23       |
| Double (double) | 64   | 1 | 11 (Bias=1023) | 52       |

Kategorie wartości

| Kategoria      | exp                          | Własności                                                                                                                                              |
|----------------|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Znormalizowane | $!= 0\dots0$<br>$!= 1\dots1$ | $E = \text{exp} - \text{Bias}$ . Ukryta jedynka: $M = 1.\text{frac}$ . Najgęściej ułożone wokół 0.                                                     |
| Zdenormaliz.   | $== 0\dots0$                 | $E = 1 - \text{Bias}$ . Brak jedynki: $M = 0.\text{frac}$ . Reprezentują +0.0 i -0.0 (frac = 0) oraz liczby bardzo bliskie zera (stopniowy underflow). |
| Specjalne      | $== 1\dots1$                 | Gdy frac == 0, to $+\infty$ lub $-\infty$ (nadmiar/dzielenie przez 0). Gdy frac $\neq$ 0, to NaN (np. $\sqrt{-1}$ , $\infty - \infty$ ).               |

Zaokrąglanie GRS i Round-to-Even

Domyślny tryb to **Round-to-Even** (zapobiega błędowi statycznemu przy sumowaniu).

... x x G    !    R S S S ...

G - ostatni zachowany, R - pierwszy usunięty, S - OR reszty

| Warunek  | Ułamek | Akcja                                       |
|----------|--------|---------------------------------------------|
| R=0      | < 0.5  | W dół (odcięcie)                            |
| R=1, S=1 | > 0.5  | W górę (+1 do G)                            |
| R=1, S=0 | = 0.5  | Do parzystej: w górę gdy G=1, w dół gdy G=0 |

Własności matematyczne i rzutowanie

- Brak łączności:**  $(a + b) + c \neq a + (b + c)$ , przez zaokrąglenia i utratę danych.
- Brak rozdzielności:**  $a(b + c) \neq ab + ac$ .
- Rzutowanie int → float:** Może stracić precyzję (zaokrągła zgodnie z trybem).
- Rzutowanie int → double:** Dokładne i bezstratne (bo 52 bity mantysy > 32 bity inta).
- Rzutowanie float/double → int:** Obcina ułamek w stronę 0. Jeśli poza zakresem lub NaN → z reguły zwraca TMin (0xB0000000).

## 7. Rejestry x86\_64

|      |      |                |      |      |                |
|------|------|----------------|------|------|----------------|
| %rax | %eax | %ax<br>%ah %al | %rbx | %ebx | %bx<br>%bh %bl |
|------|------|----------------|------|------|----------------|

|      |      |                |      |      |                |
|------|------|----------------|------|------|----------------|
| %rcx | %ecx | %cx<br>%ch %cl | %rdx | %edx | %dx<br>%dh %dl |
|------|------|----------------|------|------|----------------|

|      |      |             |      |      |             |
|------|------|-------------|------|------|-------------|
| %rsi | %esi | %si<br>%sil | %rdi | %edi | %di<br>%dil |
|------|------|-------------|------|------|-------------|

|      |      |             |      |      |             |
|------|------|-------------|------|------|-------------|
| %rbp | %ebp | %bp<br>%bpl | %rsp | %esp | %sp<br>%spl |
|------|------|-------------|------|------|-------------|

|     |      |              |     |      |              |
|-----|------|--------------|-----|------|--------------|
| %r8 | %r8d | %r8w<br>%r8b | %r9 | %r9d | %r9w<br>%r9b |
|-----|------|--------------|-----|------|--------------|

Uwaga: Rejestry od %r10 do %r15 używają schematu: %r10, %r10d, %r10w, %r10b.

Podział ról rejestrów

|                                  |                                                                                |
|----------------------------------|--------------------------------------------------------------------------------|
| %rax                             | Akumulator. Główny rejestr arytmetyczny. Przechowuje <b>wartość zwracaną</b> . |
| %rdi, %rsi, %rdx, %rcx, %r8, %r9 | Kolejno: <b>od 1. do 6. argumentu funkcji</b> . Caller-saved.                  |
| %rsp                             | Wskaźnik stosu (SP). Wskazuje wierzchołek ramki.                               |
| %rbp                             | Wskaźnik bazy (BP). Początek ramki stosu. Callee-saved.                        |
| %rbx, %r12-%r15                  | Ogólnego przeznaczenia. Wszystkie <b>callee-saved</b> .                        |
| %rip                             | Wskaźnik instrukcji (Program Counter).                                         |

## 8. Tryby adresowania (operands)

| Tryb                   | Format         | Obliczanie adresu     |
|------------------------|----------------|-----------------------|
| Natychmiastowy (Imm)   | \$Imm          | Imm                   |
| Rejestrowy (Reg)       | %Ra            | %Ra                   |
| Bezpośredni (Mem)      | Imm            | M[Imm]                |
| Pośredni (Mem)         | (%Rb)          | M[%Rb]                |
| Z przesunięciem        | D(%Rb)         | M[%Rb + DI]           |
| Skalowany (Ind/scaled) | D(%Rb, %Ri, S) | M[%Rb + %Ri * S + DI] |
| Skalowany (baseless)   | (, %Ri, S)     | M[%Ri * S]            |

Legenda: Imm / D to stała liczbowa, %Ra / %Rb to rejestr bazowy, %Ri to rejestr indeksowy, a S to skala (tylko: 1, 2, 4 lub 8).

## 9. Flagi stanu i sterowanie

|    |                                                               |
|----|---------------------------------------------------------------|
| ZF | Zero. Wynik to 0 (np. argumenty są równe).                    |
| SF | Sign. Wynik jest ujemny (MSB = 1).                            |
| CF | Carry. Przepełnienie dla liczb <b>bez znaku</b> (unsigned).   |
| OF | Overflow. Przepełnienie dla liczb <b>ze znakiem</b> (signed). |

|                          |                                                    |                                                                    |
|--------------------------|----------------------------------------------------|--------------------------------------------------------------------|
| <code>cmp S1, S2</code>  | $S2 - S1$                                          | Ustawia flagi jak odejmowanie, nie zapisuje wyniku.                |
| <code>test S1, S2</code> | $S2 \& S1$                                         | Ustawia flagi jak <b>AND</b> (np. test czy rejestr jest zerem).    |
| <code>jX dest</code>     | $\text{if}(X) \text{\%rip} \leftarrow \text{dest}$ | Skok warunkowy (X = warunek).                                      |
| <code>setX D</code>      | $\text{if}(X) D \leftarrow 1$                      | Ustawia bajt (np. <code>%al</code> ) na 0 lub 1 na podstawie flag. |
| <code>cmovX S, D</code>  | $\text{if}(X) D \leftarrow S$                      | Warunkowe kopiowanie (optymalizacja zamiast skoku).                |

Sufiksy warunkowe (dla `jX`, `setX`, `cmovX`)

| Sufiks               | Synonim | Znaczenie                               |
|----------------------|---------|-----------------------------------------|
| <code>e / z</code>   |         | Equal / Zero (równe / wynik to 0)       |
| <code>ne / nz</code> |         | Not Equal / Not Zero (nierówne)         |
| <code>s</code>       |         | Sign (wynik ujemny, <code>SF=1</code> ) |

| Liczby ze znakiem (signed)  |                 |                                   |
|-----------------------------|-----------------|-----------------------------------|
| <code>g / ge</code>         |                 | Greater / Greater or Equal        |
| <code>l / le</code>         |                 | Less / Less or Equal              |
| Liczby bez znaku (unsigned) |                 |                                   |
| <code>a / ae</code>         | <code>nc</code> | Above / Above or Equal (No Carry) |
| <code>b / be</code>         | <code>c</code>  | Below / Below or Equal (Carry)    |

Translacja struktur kontrolnych (C  $\rightarrow$  ASM)

- if-else:** Realizowane przez skoki (`jX`) lub `cmovX`. `cmovX` unika kar za błędną predykcję skoku, ale oblicza zawsze obie ścieżki. Nie używa się go, gdy operacje są kosztowne obliczeniowo lub mają efekty uboczne (np. `x++`).
- Pętla do-while:** Ciało pętli jest na początku, na końcu znajduje się test i warunkowy skok w górę (`if(war) goto loop`).
- Pętla while:**
  - Jump-to-middle** (`-Og`): początkowy skok omija ciało pętli prosto do testu na końcu.
  - Do-while conversion** (`-O1`): początkowy strażnik (`if(!war) goto done`), po którym następuje zwykła pętla typu do-while.
- Pętla for:** Kompilator konwertuje je na pętlę `while` według schematu: `init; while(war){ body; update it }`.
- Switch:** Używa **tablic skoków** dla osiągnięcia czasu skoku  $O(1)$  przy `case`. Realizowane przez pośredni skok z adresowaniem skalowanym: `jmp *.L4(,%rdi,8)`. Pominięcie `break` w języku C odpowiada fizycznemu brakowi instrukcji skoku kończącej dany blok w ASM (fall-through do kodu poniżej).

## 10. Procedury i stos

Stos w x86-64 rośnie **w dół** (w stronę niższych adresów). Rejestr `%rsp` wskazuje na **wierzchołek** stosu (najniższy zajęty adres). Wartości odkłada się używając `push` (zmniejsza `%rsp` o 8), a zdejmując `pop` (zwiększa `%rsp` o 8).

Przekazywanie sterowania

- `call dest`: Odkłada adres powrotu (kolejnej instrukcji) na stos i robi skok do `dest`.
- `ret`: Zdejmuje adres powrotu ze stosu i robi pod niego skok.

## 11. Konwencja Wywoływań (System V ABI)

`%rdi`  $\rightarrow$  `%rsi`  $\rightarrow$  `%rdx`  $\rightarrow$  `%rcx`  $\rightarrow$  `%r8`  $\rightarrow$  `%r9`

Argumenty 7+: Na stosie od końca. Wynik w `%rax`.

Rejestry caller-saved vs callee-saved

| Caller-saved                                                                                                                                | Callee-saved                                                                                                         |
|---------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| (Wołający musi zapisać)<br>Mogą zostać nadpisane w funkcji. By je zachować, caller kładzie je na stos przed <code>call</code> .             | (Wołany musi przywrócić)<br>Muszą zachować stan. Callee musi zapisać je na stos i odtworzyć przed <code>ret</code> . |
| <code>%rax</code> , <code>%rdi</code> , <code>%rsi</code> ,<br><code>%rdx</code> , <code>%rcx</code> , <code>%r8</code> - <code>%r11</code> | <code>%rbx</code> , <code>%rbp</code> , <code>%r12</code> - <code>%r15</code>                                        |

Ramka stosu

Każde wywołanie funkcji otrzymuje własną ramkę na stosie (dzięki temu **rekurencja** działa naturalnie). **Alokacja ramki jest potrzebna, gdy:** brakuje rejestrów na zmienne (spilling), użyto lokalnej tablicy/struktury, lub pobrano adres zmiennej lokalnej (operator `&`).

| Prolog                                                                                                                                       | Epilog                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <pre>pushq %rbp ;<br/>Zapisz stary base ptr<br/>movq %rsp, %rbp ;<br/>Nowy base ptr = rsp<br/>subq \$32, %rsp ;<br/>Alokacja 32 bajtów</pre> | <pre>addq \$32, %rsp ;<br/>Zwolnij alokację<br/>popq %rbp ;<br/>Przywróć base ptr<br/>ret ;<br/>Skok powrotny</pre> |

**Sprzątanie przez `leave`:** Zastępuje `movq %rbp, %rsp` i `popq %rbp` w jednej instrukcji.

## 12. Katalog instrukcji (AT&T)

| Opcode                      | Efekt                         | Opis                                         |
|-----------------------------|-------------------------------|----------------------------------------------|
| <code>mov S, D</code>       | $D \leftarrow S$              | Kopiuje wartość z S do D.                    |
| <code>lea S, D</code>       | $D \leftarrow \text{addr}(S)$ | Oblicza adres S (bez czytania pamięci).      |
| <code>add S, D</code>       | $D \leftarrow D + S$          | Dodawanie (ustawia flagi).                   |
| <code>sub S, D</code>       | $D \leftarrow D - S$          | Odejmowanie (ustawia flagi).                 |
| <code>imul S, D</code>      | $D \leftarrow D * S$          | Mnożenie liczb ze znakiem.                   |
| <code>sal / shl k, D</code> | $D \ll= k$                    | Przesunięcie w lewo (mnożenie przez $2^k$ ). |

| Opcode                     | Efekt                     | Opis                                                          |
|----------------------------|---------------------------|---------------------------------------------------------------|
| <code>sar k, D</code>      | $D \gg = k$               | Arytmetyczne w prawo (dzielenie). <b>Kopiuje znak</b> .       |
| <code>shr k, D</code>      | $D \gg = k$               | Logiczne w prawo. Dopełnia zerami.                            |
| <code>and / or S, D</code> | $D \leftarrow D \& /   S$ | Operacje bitowe (ustawiają flagi).                            |
| <code>xor S, D</code>      | $D \leftarrow D \wedge S$ | Często używane jako <code>xor %rax, %rax</code> do zerowania. |

#### Porównania i sterowanie

|                          |                                                    |                                                       |
|--------------------------|----------------------------------------------------|-------------------------------------------------------|
| <code>cmp S1, S2</code>  | $S2 - S1$                                          | Ustawia flagi jak odejmowanie, nie zapisuje wyniku.   |
| <code>test S1, S2</code> | $S2 \& S1$                                         | Ustawia flagi jak <b>AND</b> .                        |
| <code>jX dest</code>     | $\text{if}(X) \text{\%rip} \leftarrow \text{dest}$ | Skok warunkowy (X = warunek).                         |
| <code>setX D</code>      | $\text{if}(X) D \leftarrow 1$                      | Ustawia najmłodszy bajt na 0 lub 1 na podstawie flag. |
| <code>cmovX S, D</code>  | $\text{if}(X) D \leftarrow S$                      | Warunkowe kopiowanie (optymalizacja zamiast skoku).   |

#### Stos i zarządzanie procedurami

|                        |                                                                 |                                                       |
|------------------------|-----------------------------------------------------------------|-------------------------------------------------------|
| <code>push S</code>    | $\text{\%rsp} - = 8$<br>$M[\text{\%rsp}] \leftarrow S$          | Odkłada na stos (zmniejsza <code>%rsp</code> ).       |
| <code>pop D</code>     | $D \leftarrow M[\text{\%rsp}]$<br>$\text{\%rsp} + = 8$          | Zdejmuje ze stosu do D (zwiększa <code>%rsp</code> ). |
| <code>call dest</code> | push <code>%rip</code><br>$\text{\%rip} \leftarrow \text{dest}$ | Skok do funkcji (zapisuje adres powrotu na stosie).   |
| <code>ret</code>       | pop <code>%rip</code>                                           | Zdejmuje adres powrotu ze stosu i skacze pod niego.   |
| <code>leave</code>     | $\text{\%rsp} \leftarrow \text{\%rbp}$<br>pop <code>%rbp</code> | Sprząta ramkę stosu (odwrotność prologu).             |

**Uwaga o sufiksach:** Instrukcje przyjmują sufiks określający rozmiar danych: `b` (8-bit), `w` (16-bit), `l` (32-bit), `q` (64-bit).  
Np. `movl` kopiuje 32 bity (i automatycznie zeruje górną połowę 64-bitowego rejestru!).